

Cybersecurity

Ifs and Lists

Lesson 6.2



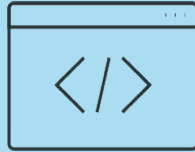
Learning Objectives

By the end of today's class, you will be able to:

- 1 Read Bash and interpret scripts.
- 2 Use variables in Bash scripts.
- 3 Use `if` statements in Bash scripts.
- 4 Use lists in Bash scripts.
- 5 Iterate through lists with a `for` loop.



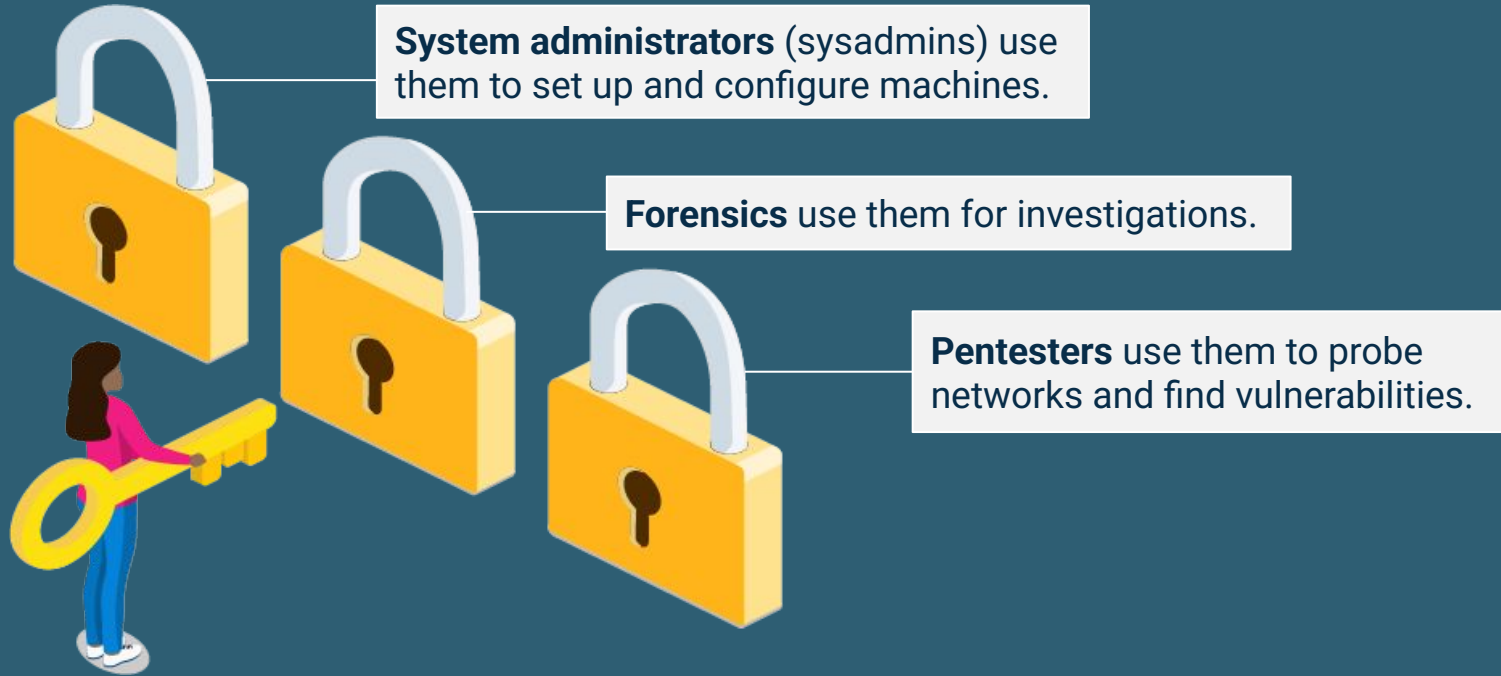
Welcome



If Statements

Bash Scripting

Bash scripting is a vital skill for any cybersecurity professional. Scripts can be used for the following:



Scripting Skills

Today, we'll continue to develop our scripting skills to incorporate the following into our script:



if statements to control flow based on conditions



for loops to complete repetitive tasks



To automate the setup of a machine



Criteria and Decision-Making

We will need our scripts to make decisions based on specific criteria



Our scripts can **check** for:

- Users
- Directories
- Files
- Permissions

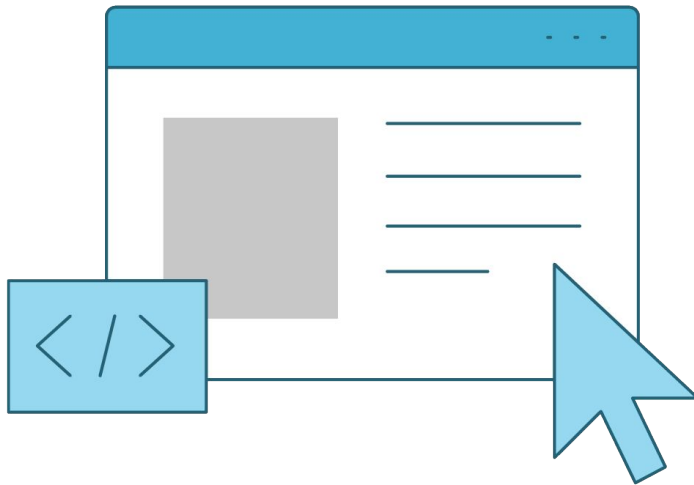
Based on the results, the script can take a specific action.



Criteria and Decision-Making

To accomplish these tasks, we will need to learn a few scripting techniques:

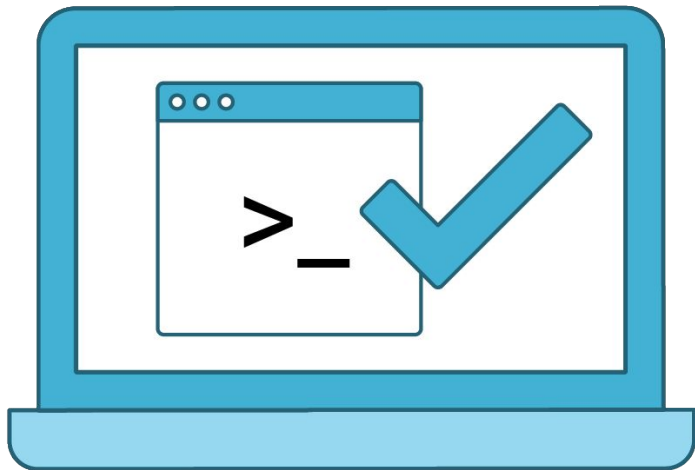
- Using `if` statements
- Using `if/else` statements
- Comparing multiple conditions using `if/else` statements





Criteria and Decision-Making

We will need our scripts to make decisions based on specific criteria.



`if` is used in our scripts as part of a **conditional statement**.

Based on a **specific condition** (or criterion), the script will take a certain action:

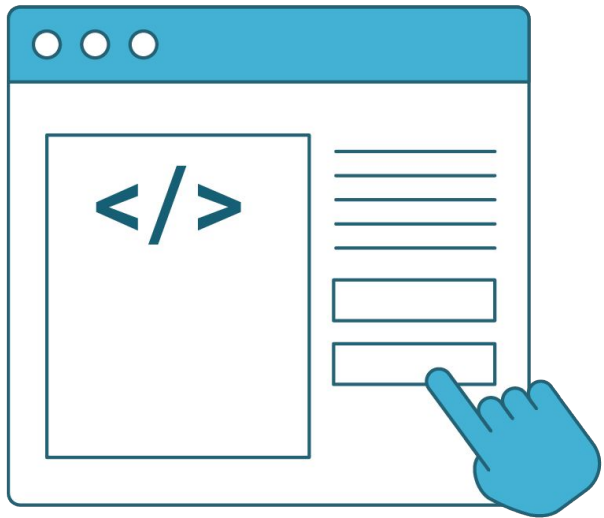
- **If** this command runs, **then** exit the script.
- **If** this file exists, **then** run that command.



Demonstration Scenario

We will give a quick overview of each technique and then provide a few specific demonstrations.

We will also encounter **comments** and the **exit** command.





Comments

Bash Scripting

```
# This is a comment
```



A comment is a **non-executable** line of text.



Helps the developers remember what their script is supposed to do.



Place a **#** in front of any line we want to be a comment.



if Syntax

Bash Scripting

```
if [ <test> ]  
then  
<run_this_command>  
fi
```

if	Initiates our if statement
[]	Encapsulates the condition
then	Then runs following commands if the condition is met
fi	Ends the if statement



if Syntax

Bash Scripting

```
if [ 5 -gt 8 ]  
then  
    echo "This doesn't make sense!"  
fi
```

<code>if [5 -gt 8]</code>	This will check if 5 is greater than 8.
<code>then</code>	This runs the following commands if the condition is met.
<code>echo "This doesn't make sense!"</code>	This will have the script print to the screen "That doesn't make sense!"
<code>fi</code>	This ends the if statement.



if/else Syntax

Bash Scripting

```
if [ <test> ]  
then  
    <run_this_code>  
else  
    <run_this_code>  
fi
```

<code>if</code> [<condition>]	If this test is true...
<code>then</code>	Runs the following code
<code>else</code>	Runs the following code if the condition is false
<code>fi</code>	Ends the if statement



if/else Syntax

Bash Scripting

```
if [ 5 -gt 4 ]  
then  
    echo "That is correct!"  
else  
    echo "That doesn't make sense!"  
fi
```

<code>if [5 -gt 4]</code>	This will check if 5 is greater than 4.
<code>then</code>	Runs following code if the condition is met
<code>echo "That is correct!"</code>	Is printed to the screen
<code>else</code>	Runs the following code if the condition is unmet: <code>echo "This doesn't make sense!"</code>
<code>fi</code>	Ends the if statement



Does anyone remember
how a **string of commands**
separated by ; or && is
affected?



&& Syntax

Bash Scripting

```
if [ <condition_1> ] && [ <condition_2> ]  
then  
    <run_this_code>  
fi
```

<code>if [<test1>]</code>	Checks if one condition is true
<code>&& [<test2>]</code>	Checks if a second condition is true
<code>then</code>	Runs the following code if both conditions are met
<code>fi</code>	Ends the if statement



&& Syntax

Bash Scripting

```
if [ 5 -gt 4 ] && [ 4 -gt 3 ]  
then  
    echo "That makes sense."  
fi
```

If the following two conditions are met:

```
if [ 5 -gt 4 ]
```

```
&& [4 -gt 3]
```

Run **echo "That makes sense."**



|| Syntax

Bash Scripting

```
if [ <condition1> ] || [ <condition2> ]  
then  
  <run_this_code>  
else  
  <run_this_code>  
fi
```

<code>if [<condition1>]</code>	If condition 1 is true
<code> [<condition 2>]</code>	Or if condition 2 is true
<code>then</code>	Run the following code
<code>else</code>	Runs code if both conditions are false
<code>fi</code>	Ends the if statement



|| Syntax

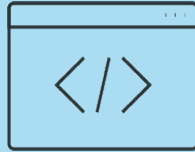
Bash Scripting

```
if [5 -gt 4] || [4 -gt 3 ]
then
    echo "That only partially makes sense."
else
    echo "None of this makes sense."
fi
```

if [5 -gt 4]	If this condition is true...
 [4 -gt 3]	...or if this test is true...
then	Run the following code.
else	Otherwise (if both conditions are false), run the following command.
fi	Ends the if statement.

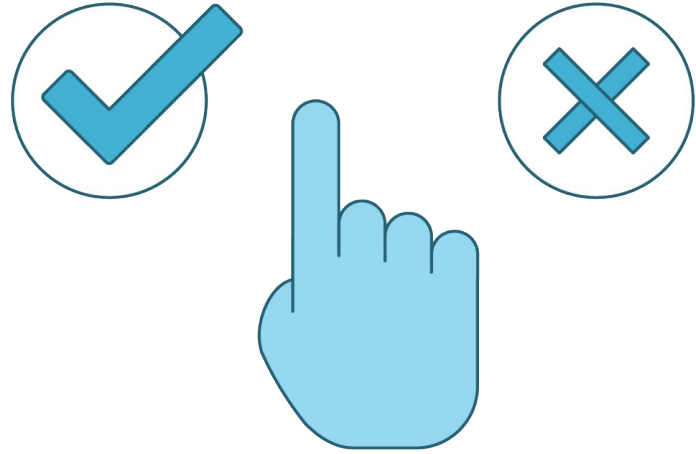
Demo Summary

if	<pre>if [<condition>] then <run_this_command> fi</pre>	Runs code if the condition is met.
if / else	<pre>if [<condition>] then <run_this_command> else <run_this_command> fi</pre>	Runs code if the condition is met. If condition isn't met, it will run a different command.
&&	<pre>if [<condition1>] && [<condition2>] then <run_this_command> fi</pre>	Runs code if more than one condition is met.
	<pre>if [<condition1>] [<condition2>] then <run_this_command> else <run_this_command> fi</pre>	Runs code if only one of multiple conditions are met.



Creating **Conditionals**

We will now cover how to design the conditions that are ruled **true** or **false**.





Creating Conditions

Comparing Values with Bash Conditional Expressions

<code>==</code>	This string is equal to another
<code>!=</code>	If two strings are not equal
<code>-gt</code>	If one integer is greater than another
<code>-lt</code>	If one integer is less than another
<code>-d /path_to/directory</code>	Checks for the existence of a directory
<code>-f /path_to/files</code>	Checks for the existence of a file



Creating Conditions

Equals to

```
# If $x is equal to $y, run the echo command.  
if [ $x = $y ]  
then  
    echo "X is equal to Y!"  
fi
```

<code>if [\$x = \$y]</code>	If the value of the x variable is equal to the value of the y variable
<code>then</code>	Then run the following command
<code>echo "X is equal to Y!"</code>	The echo command that will run if the initial condition is met
<code>fi</code>	Ends the if statement



Creating Conditions

Not equals to

```
# If $x is not equal to $y, run the echo command.  
if [ $x != $y ]  
then  
    echo "X does not equal Y!"  
fi
```

`if [ $x != $y ]`

If the value of the x variable is not equal to the value of the y variable

`then`

Then run the following command

`echo "X does not equal Y!"`

The echo command that will run if the initial condition is met

`fi`

Ends the if statement



Creating Conditions

Conditionals and Strings

```
# If str1 is not equal to str2, run the echo command and exit the script.
if [ "$str1" != "$str2" ]
then
    echo "These strings do not match."
    echo "Exiting this script."
fi
```

<code>if ["\$str1" != "\$str2"]</code>	If string 1 is not equal to string 2...
<code>then</code>	Then run the following command.
<code>echo "These strings do not match"</code>	
<code>echo "Exiting this script."</code>	
<code>fi</code>	Ends the if statement.



Creating Conditions

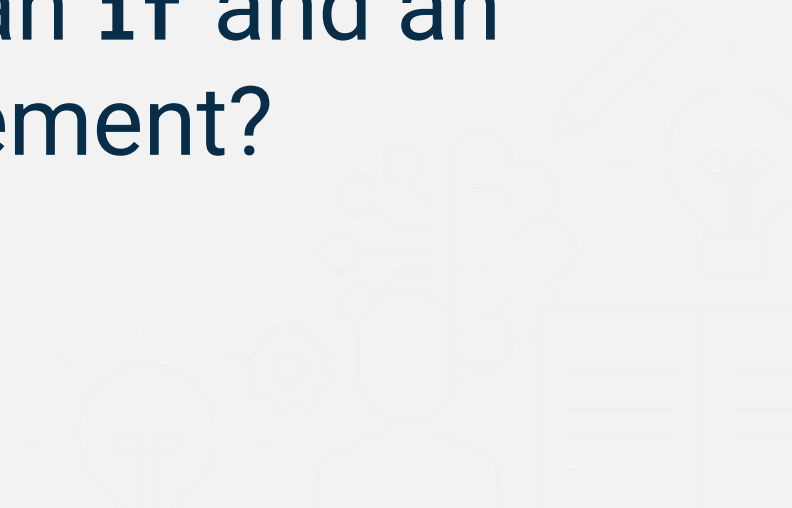
Greater Than and Less Than

```
# If x is greater than y, run the echo command
if [ $x -gt $y ]
then
    echo "$x is greater than $y".
fi
```

```
# Checks if x is less than y
if [ $x -lt $y ]
then
    echo "$x is less than $y!"
else
    echo "$x is not less than $y!"
fi
```



What's the difference
between an **if** and an
else statement?



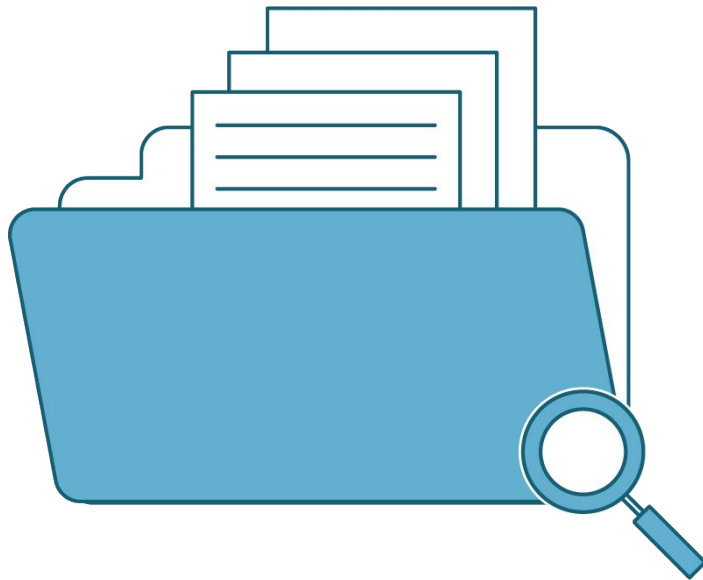
Checking Files and Directories

Check for files and directories using the following options:

-f checks for files



-d checks for directories





Checking Files and Directories

```
# check for the /etc directory
if [ -d /etc ]
then
    echo The /etc directory exists!
fi
```

```
# check for my_cool_folder
if [ ! -d /my_cool_folder ]
then
    echo my_cool_folder is not there!
fi
```

```
# check for my_file.txt
if [ -f /my_file.txt ]
then
    echo my_file.txt is there
fi
```

```
if [ -d /etc ]...
```

If the **/etc** directory exists, run the following echo command.

```
if [ ! -d /my_cool_folder ]...
```

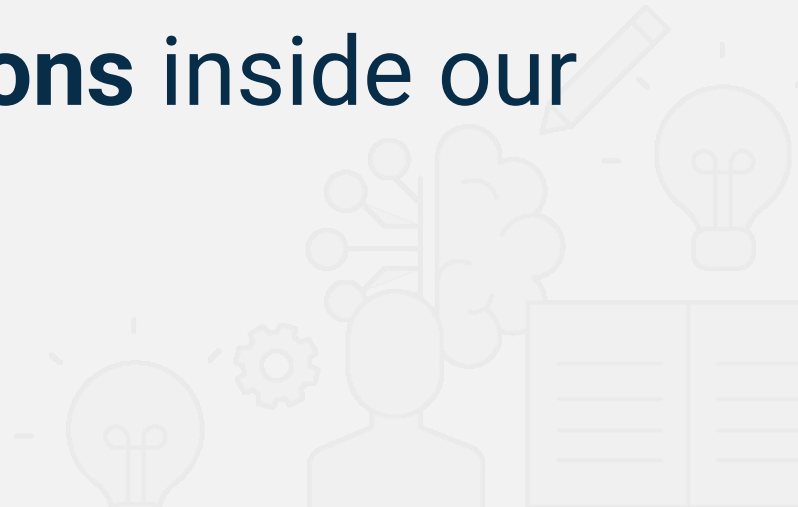
If **/my_cool_folder** does not exist, run the following echo command.

```
if [ -f /my_file.txt ]...
```

If the file **/my_file.txt** exists, then run the following echo command.



We can use **built-in variables** and **command expansions** inside our tests.



Built-In Variables and Command Expansions

We will use these variables and command expansions to verify:



Username



UID



**User running
the current
script**



Built-In Variables and Command Expansions

Checking Username

```
# if the user who ran this script is not the sysadmin user, run the echo command
```

```
if [ $USER != 'sysadmin' ]
```

```
then
```

```
    echo "You are not the sysadmin!"
```

```
    exit
```

```
fi
```

```
if [ $USER != 'sysadmin' ]
```

If the user who ran this script is not “**sysadmin**,” then run the following echo command.



Built In Variables and Command Expansions

Checking UID

```
# if the uid of the user who ran this script does not equal 1,000, run the  
echo command
```

```
if [ $UID != 1000 ]
```

```
then
```

```
    echo your UID is wrong
```

```
    exit
```

```
fi
```

```
if [ $UID != 1000 ]
```

If the **UID** of the user who is running this script does not equal 1,000, then run the following echo command.



Built-In Variables and Command Expansions

Checking User Running the Script

```
# if the sysadmin ran the script, run the echo command
if [ $(whoami) = 'sysadmin' ]
then
    echo 'you are the sysadmin!'
fi
```

```
if [ $(whoami) = 'sysadmin' ]
```

If the user “sysadmin” ran the script, then run the following echo command.



Activity:

Variables and If Statements

In the previous class, you created your first Bash script, edited `$PATH` variables to create custom commands, and added aliases to your `~/ .bashrc` file.

- Now, we will add variables and conditional if statements to our scripts.

Suggested Time:
20 minutes



Time's up!
Let's review



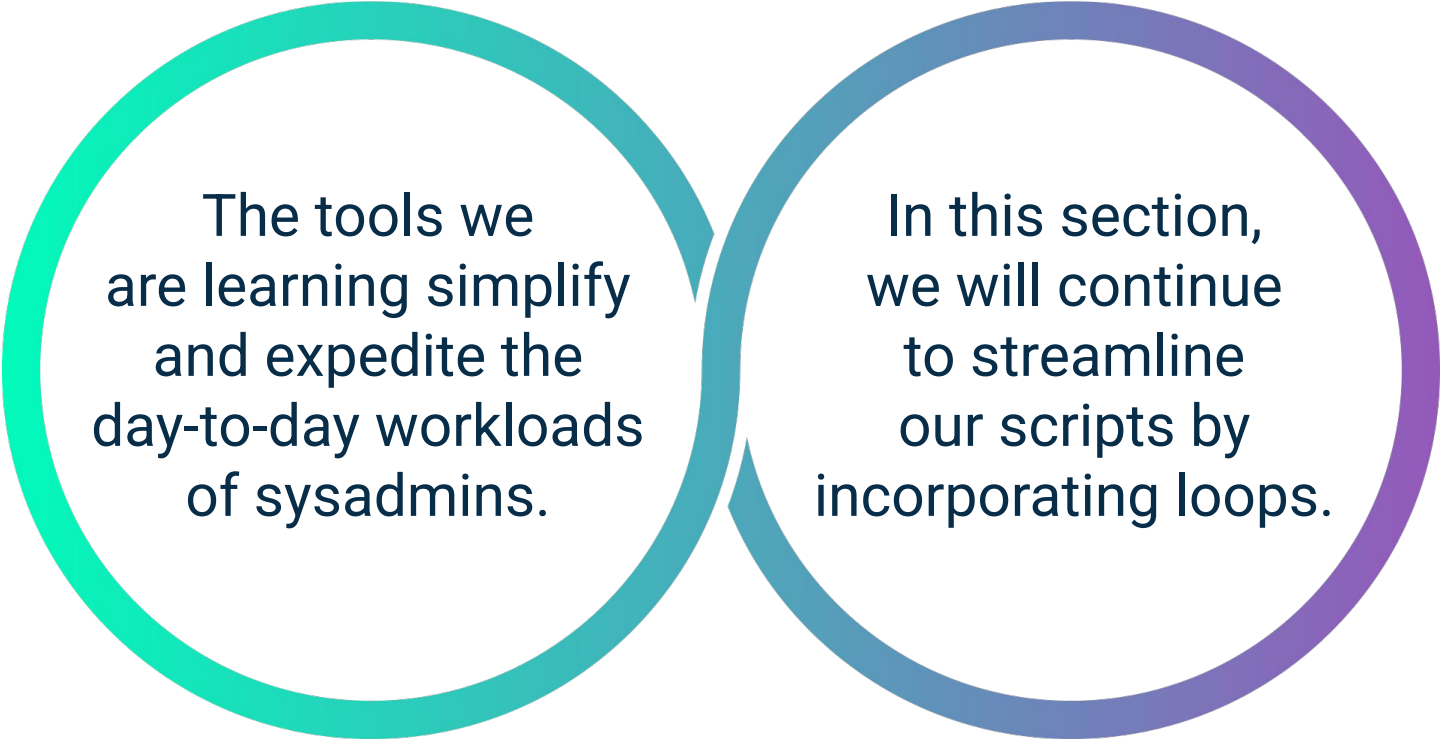
Questions?





Lists & Loops

Optimizing our Scripts

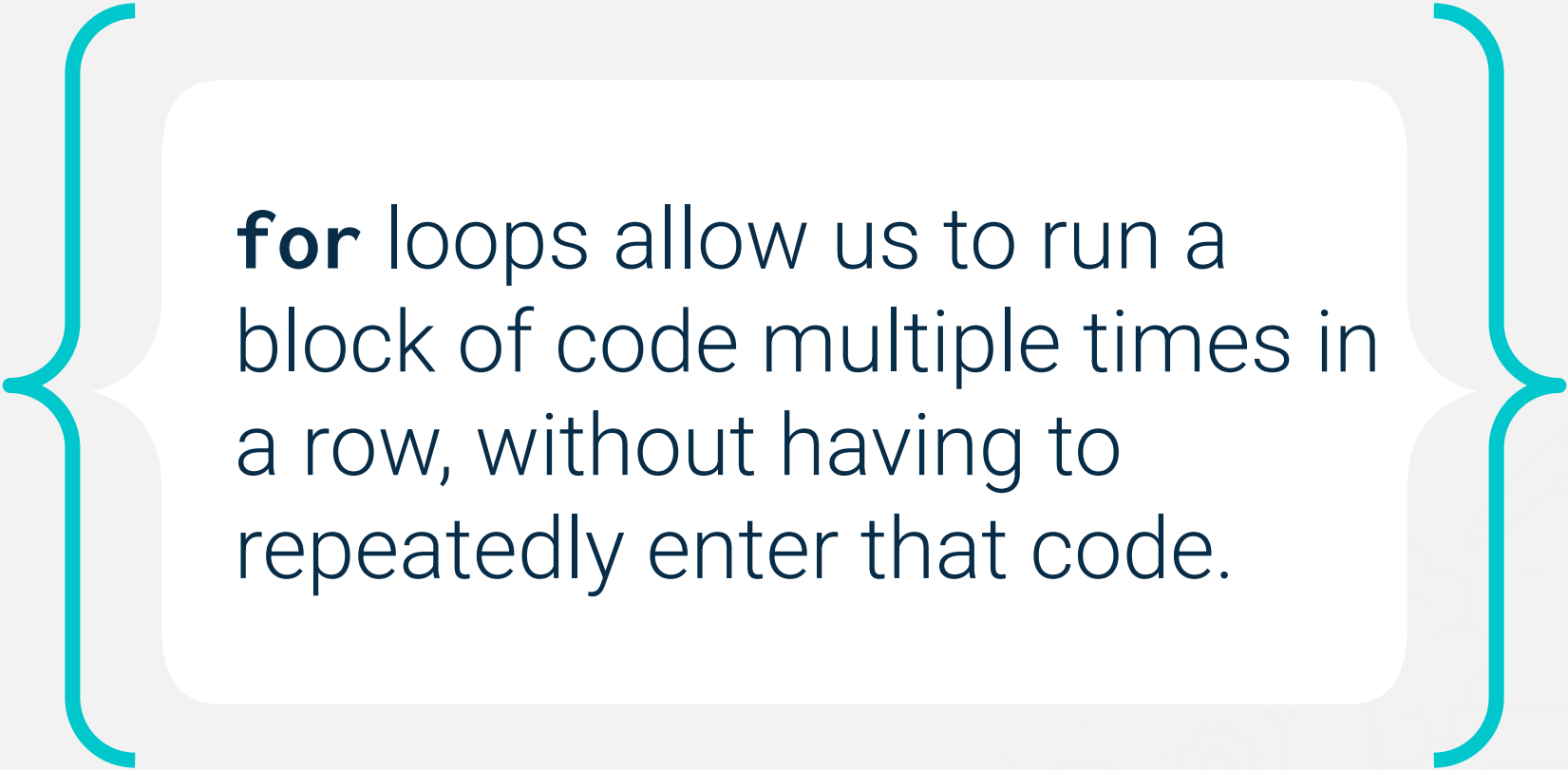


The tools we
are learning simplify
and expedite the
day-to-day workloads
of sysadmins.

In this section,
we will continue
to streamline
our scripts by
incorporating loops.

Sysadmins will often
complete the same
common task over and
over and over again.





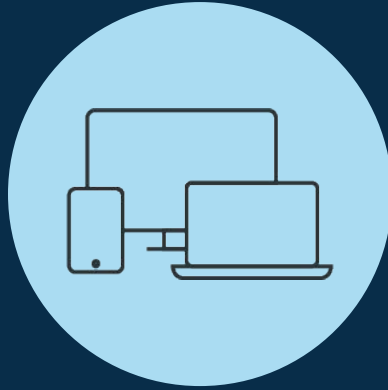
for loops allow us to run a block of code multiple times in a row, without having to repeatedly enter that code.



for Loops

Code is run against each item in a list of items:

```
for package in ${packages[@]}  
do  
    if [ ! $(which $package) ]  
    then  
        apt install -y $package  
    fi  
done
```



Instructor Demonstration

for Loops



Demo Summary

List and Loops Syntax

Made lists

```
my_list=(a b c d e f)
```

Created for loops

```
# for <item> in <list>;  
# do  
#   <run_this_command>  
#   <run_this_command>  
# done
```

Accessed the list with commands

```
$ echo ${my_list[0]}  
a  
$ echo ${my_list[4]}  
e  
$ echo ${my_list[@]}  
a b c d e f
```

Created loops with conditionals

```
# run an operation on each number  
for num in {0..5};  
do  
    if [ $num = 1 ] || [ $num = 4 ]  
    then  
        echo $num  
    fi  
done
```



Activity:

Lists and Loops

In the previous activity, we added variables to our script and used if statements to check various conditions.

- Next, we will use loops to automate repetitive tasks.

Suggested Time:
20 minutes



Time's up!
Let's review

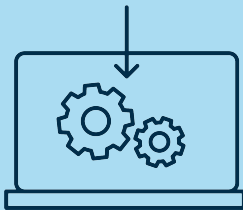


Questions?





Break
15 mins



for Loops for
a **System Administrator**



Writing a script might be overkill for the scale of the task, so we can use **for** loops directly on the command line.

For Loops for SysAdmins

Uses of loops that are extremely valuable to cybersecurity professionals:

01

List of packages



02

Results of a find command



03

List of files, check their permissions, and change

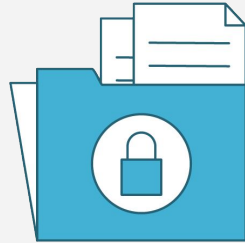


For Loops for SysAdmins

Uses of loops that are extremely valuable to cybersecurity professionals:

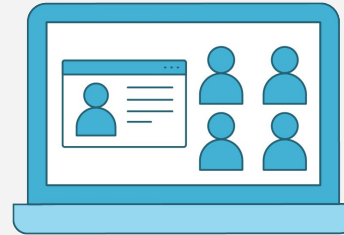
05

**List of files and create
a cryptographic hash**



06

**All the users on the
system and take an
action**

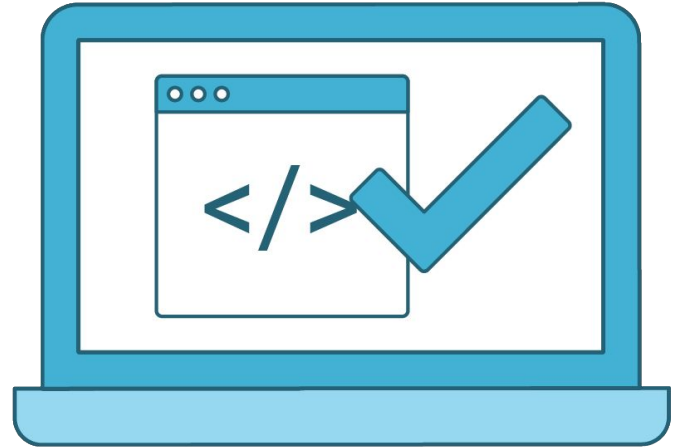




Demonstration Scenario

We will use loops to:

- Run through a list of packages and check if certain ones are installed.
- Search users' home directories for scripts and print a confirmation statement.
- Loop through scripts in your scripts folder and change the permissions to execute.
- Create a for loop that moves through a group of files and creates a hash of each file.





Questions?





Activity:

Loops for System Administrators

In the previous activity, we created for loops. Now, we will use those loops to complete tasks that are useful to a system administrator.

- In this activity, you will add loops to your `sys_info.sh` script, and run loops directly on the command line.

Suggested Time:
25 minutes



Time's up!
Let's review



Questions?





Optional: Script Along Setup

- So far, we have used if statements to control the execution flow in a script.
- We also used if statements with loops to perform administrative tasks.
- For our final activity, we apply these new skills by going through a completed user setup script.



Next class

Linux Scavenger Hunt



Thank you